**XU EXPONENTIAL UNIVERSITY OF APPLIED SCIENCES**

# Character-Level versus Word-Level Neural Text Generation: A Comparative Case Study Using the World War II Wikipedia Corpus

Seminar Paper

[Course Name]

[Semester, e.g., Summer Semester 2026]

Submitted to: [Name of Lecturer]

Author: [Full Name]

Matriculation Number: [XXXXXXX]

Date of Submission: [DD Month YYYY]

# Table of Contents

1. Introduction....4
2. Theory: State of Research and Academic Debate....5
2.1 From Statistical to Neural Language Models....5
2.2 Tokenization Strategies: Characters, Words, and Sub-words....6
2.3 Recurrent Neural Networks for Sequence Modeling....7
2.4 The Transformer Architecture and Self-Attention....8
2.5 Granularity Trade-offs in the Empirical Literature....9
2.6 Low-Resource and Small-Corpus Generative Modeling....10
2.7 Historical Encyclopedic Text as a Modelling Domain....11
3. Methodology or Use Case....12
3.1 Research Design and Data Collection....12
3.2 Dual Preprocessing Pipelines....12
3.3 Model Architectures....13
3.4 Training Procedure....13
3.5 Evaluation Approach....14
4. Analysis & Discussion....15
4.1 Convergence Behaviour....15
4.2 Qualitative Analysis: Character-Level Models....16
4.3 Qualitative Analysis: Word-Level Models....16
4.4 Memorisation versus Generalisation....17
4.5 Comparing Tokenization Schemes Directly....18
4.6 Architectural Effects Within Each Scheme....18
5. Conclusions....20
6. References....22
7. Appendix/Annex....24

## Abstract

This paper investigates how tokenization granularity interacts with model architecture when training small, from-scratch neural language models on a single, domain-specific historical text. Using a condensed, roughly 13,200-character excerpt of the English Wikipedia article on World War II, four generative models are trained: a character-level and a word-level tokenization scheme, each paired with a vanilla recurrent neural network (RNN) and a small causal Transformer encoder. All four models are trained under identical optimisation settings and compared on training-loss convergence and on text generated from the shared prompt “Germany invaded.” The theoretical background traces the development of neural language modelling from n-gram statistics through recurrent architectures to self-attention, and situates the character-versus-word question within the broader literature on tokenization and low-resource text generation. The results show that character-level models remain orthographically legible — producing plausible English letter sequences — even when the content is semantically incoherent, because the output space is restricted to roughly seventy symbols. Word-level models instead reproduce locally coherent phrases lifted almost verbatim from the training text but frequently fail to combine them into grammatical sequences, reflecting the much larger and sparser vocabulary they must learn from so little data. Both effects are attributed primarily to memorisation rather than generalisation, given the small corpus size. The paper concludes with implications for choosing a tokenization strategy under severe data scarcity and outlines directions for extending the experiment to the full-length article and larger training budgets.

# 1. Introduction

Large language models are, at their core, systems that predict the next unit of text given the units that came before. A design decision that shapes everything downstream of that simple objective is the granularity of the unit itself: should a model predict one character at a time, one word at a time, or something in between, such as a sub-word piece? This choice interacts with the model architecture used to make the prediction, and both interact with how much training data is available. Modern large-scale systems side-step much of this tension by combining sub-word tokenization with billions of training tokens, but the underlying trade-offs re-emerge sharply whenever a model must be trained from scratch on a small, domain-specific corpus — a situation common in specialised research, low-resource languages, and educational settings. This paper uses a controlled, from-scratch experiment to examine that trade-off directly. A single historical text — a condensed excerpt of the English Wikipedia article on World War II, roughly 13,200 characters long — is tokenized in two ways, at the character level and at the word level, and each tokenization is paired with two architectures, a vanilla recurrent neural network (RNN) and a small causal Transformer encoder. All four resulting models are trained under identical optimisation settings on the same underlying text, which isolates the effect of tokenization granularity and architecture from confounds such as differences in data, training duration, or objective function.

The central argument advanced here is that under conditions of severe data scarcity, character-level and word-level models fail in systematically different, almost complementary ways. Character-level models, because they only ever choose among a small alphabet of symbols (roughly seventy in this corpus), remain orthographically legible: their output looks like English, with plausible spelling and word boundaries, even when the content drifts into nonsense. Word-level models, by contrast, draw from a much larger and sparser vocabulary (roughly 740 distinct tokens here) and tend to reproduce short, locally coherent phrases copied almost verbatim from the source text, but they struggle to connect those phrases into grammatically sound longer sequences. Both behaviours are best explained not as

generalised language competence but as memorisation of a small training set, a distinction the paper returns to repeatedly.

The remainder of the paper is structured as follows. Chapter 2 reviews the theoretical and empirical literature relevant to the experiment: the historical progression from n-gram models to neural sequence models, the specific mechanics of RNNs and the Transformer's self-attention mechanism, existing findings on character- versus word-level (and sub-word) tokenization, and the particular challenges of training generative models on small, low-resource corpora. Chapter 3 describes the methodology in detail, covering data collection, the two preprocessing pipelines, the two model architectures, and the training and evaluation procedure. Chapter 4 presents and discusses the results, comparing convergence behaviour across the four training runs and analysing the qualitative properties of text each model generates from a shared prompt. Chapter 5 concludes by tying the findings back to the central argument, acknowledging the experiment's limitations, and suggesting directions for future work, such as scaling to the full-length article or introducing sub-word tokenization as a middle ground between the two extremes studied here.

# 2. Theory: State of Research and Academic Debate

## 2.1 From Statistical to Neural Language Models

Language modelling — assigning a probability to a sequence of symbols, or equivalently predicting the next symbol given its predecessors — predates deep learning by decades. Classical n-gram models estimated the probability of a token from the frequency of its preceding $n-1$ tokens in a training corpus, an approach that is simple and interpretable but suffers badly from data sparsity: most plausible sequences of even moderate length are never observed in the training data, forcing heavy reliance on smoothing techniques. N-gram models are also fundamentally local, unable to capture dependencies that span more tokens than the chosen order n, which makes them poorly suited to modelling long-range structure such as the thematic and referential coherence found in encyclopedic prose.

The shift to neural language models replaced discrete frequency counting with continuous, learned representations. Instead of storing exact counts for every observed n-gram, a neural model learns a distributed embedding for each vocabulary item and a parametric function that predicts the next item's probability distribution from the embeddings of the context. This allows the model to generalise across similar contexts — two sequences that are not identical but semantically or syntactically similar can still receive similar predictions, something an n-gram table cannot do. Early neural language models used feed-forward networks over a fixed-size context window, but the fixed window reintroduced a version of the n-gram order problem: dependencies beyond the window remained invisible to the model.

### 2.2 Tokenization Strategies: Characters, Words, and Sub-words

The unit over which a language model operates — its tokenization — is a foundational design choice with consequences for vocabulary size, sequence length, and the kind of structure the model must learn. Word-level tokenization treats whitespace- and punctuation-delimited word forms as atomic units. It has the advantage of producing short sequences relative to the underlying text and units that already carry substantial lexical meaning, but it produces large vocabularies (in this study, 742 distinct tokens from a 13,200-character excerpt) and cannot, by construction, generate or represent a word form absent from the training vocabulary. Rare words, proper nouns, and morphological variants are especially exposed to this out-of-vocabulary problem, which is a serious concern for a historical text dense with country names, dates, and military terminology.

Character-level tokenization instead treats individual characters as the atomic unit. Its vocabulary is small and closed (74 symbols here, corresponding to the letters, digits, punctuation, and space characters present in the excerpt), so the model can in principle produce any string composed of those characters, including words never seen during training, simply by generating a plausible sequence of letters. The cost is that sequences become far longer for a given amount of text — 13,197 characters versus 2,473 word tokens for the same excerpt in this study, a factor of roughly 5.3 — which places a heavier burden on the model to

retain information over long spans and increases the number of prediction steps needed to produce coherent multi-word output.

Sub-word tokenization schemes, such as byte-pair encoding, were developed as a compromise between these two extremes. By merging frequently co-occurring character pairs into progressively larger units, sub-word tokenizers produce a vocabulary of a chosen intermediate size that can represent common words as single tokens while still being able to fall back to smaller pieces, or ultimately individual characters, for rare or unseen words. This closed-vocabulary, open-text property is one reason sub-word tokenization became the default choice in most large-scale neural language models built after the mid-2010s. The present study deliberately does not use sub-word tokenization, in order to isolate and contrast the two extremes of the granularity spectrum rather than study the compromise directly; this choice, and its implications, are revisited in the concluding chapter.

### 2.3 Recurrent Neural Networks for Sequence Modeling

Recurrent neural networks address the fixed-window limitation of feed-forward language models by maintaining a hidden state that is updated at every time step and, in principle, carries information from arbitrarily far back in the sequence. At each step, the network combines the embedding of the current input token with the hidden state inherited from the previous step to produce both an updated hidden state and a prediction for the next token. This recurrent update rule allows a single set of parameters to be applied uniformly regardless of sequence length, and it was the dominant architecture for sequence modelling for roughly two decades.

In practice, vanilla RNNs of the kind used in this study are known to struggle with long-range dependencies because of the vanishing and exploding gradient problem: as gradients are propagated backward through many time steps during training, they tend to shrink toward zero or grow without bound, making it difficult for the network to learn relationships between tokens that are far apart in the sequence. Gated variants such as the Long Short-Term Memory (LSTM) network and the Gated Recurrent Unit (GRU) were introduced specifically to mitigate

this problem by adding learned gating mechanisms that control what information is retained, forgotten, or written to the hidden state at each step. The present study intentionally uses a plain (non-gated) RNN rather than an LSTM or GRU, which is a relevant methodological choice: it means any weaknesses attributable to long-range dependency modelling are likely to be more pronounced here than they would be with a gated recurrent architecture of comparable size, a point that becomes relevant when interpreting the generated samples in Chapter 4.

## 2.4 The Transformer Architecture and Self-Attention

The Transformer architecture replaced recurrence with self-attention, a mechanism that lets every position in a sequence attend directly to every other position in a single computational step, rather than propagating information sequentially through a chain of hidden states. Each token's representation is updated as a weighted combination of all other tokens' representations, with the weights computed from a learned compatibility function between queries and keys. Because this computation does not depend on a previous hidden state, it can be parallelised efficiently across an entire sequence during training, and it does not suffer from the vanishing-gradient degradation associated with propagating information through many sequential recurrent steps.

A causal (autoregressive) Transformer, as used in this study, restricts each position's attention to only the positions at or before it, implemented via a triangular attention mask, so that the model cannot see future tokens when predicting the current one — the same left-to-right constraint that recurrence enforces implicitly. Because self-attention itself is permutation-invariant (it has no inherent notion of token order), Transformers require an explicit positional signal; the implementation used here adds a learned positional embedding to each token's representation before the attention layers, rather than the fixed sinusoidal encoding proposed in the original Transformer paper. Multi-head attention, in which several attention computations are performed in parallel with different learned projections and then combined, allows the

model to attend to different kinds of relationships (for instance, syntactic adjacency versus longer-range thematic association) simultaneously.

Self-attention's chief theoretical advantage over recurrence is a shorter path length between any two tokens in the sequence: a direct, one-step connection versus a chain of recurrent updates whose length grows with the distance between the tokens. This is generally expected to make Transformers better than vanilla RNNs at capturing long-range dependencies, particularly at the character level, where meaningful dependencies (for example, matching an opening and closing quotation mark, or maintaining subject–verb agreement across an intervening clause) can span many tokens. However, this advantage is most clearly realised with large training corpora; with only a few thousand training examples of a fixed, short sequence length, as in the present study, the theoretical benefit of self-attention may be substantially diminished, since there is little data from which to learn which long-range relationships matter.

### 2.5 Granularity Trade-offs in the Empirical Literature

Empirical comparisons of character- and word-level neural language models generally converge on a similar pattern to the one this paper investigates. Character-level models tend to produce output that is locally well-formed — correctly spelled words, plausible morphology — because the model only ever has to choose among a small, closed set of symbols at each step, and because the character sequence itself enforces grammatical regularities such as consistent word length and common letter combinations. Their disadvantage is that meaningful semantic or syntactic structure typically only emerges over many more prediction steps than an equivalent word-level model would need, since each character carries far less information than a whole word; character-level models therefore generally need either much more training data or much larger and more expressive networks (or both) to model the same span of meaning that a word-level model captures in a handful of tokens.

Word-level models, in the same literature, are reported to be locally more coherent per token — a single predicted token is already a real, meaningful word, and short phrases can look fluent

— but their much larger and more sparsely populated vocabulary makes them considerably more data-hungry: many words in any given vocabulary will be seen only a handful of times in a small corpus, giving the model little basis for learning how they combine with other words in novel contexts. This tends to manifest as word-level models reproducing memorised chunks of the training text with reasonably good local grammar, while struggling to generalise to combinations of words that were not seen together during training — precisely the pattern this paper's results reproduce and analyse in Chapter 4.

### 2.6 Low-Resource and Small-Corpus Generative Modeling

A separate but related strand of research addresses what happens when any neural language model, regardless of tokenization, is trained on a small corpus rather than the web-scale datasets typical of modern practice. With too little data relative to a model's capacity, a well-known failure mode is that the model effectively memorises the training set rather than learning generalisable structure: at generation time, it reproduces near-verbatim excerpts of what it has already seen instead of producing genuinely novel but plausible continuations. This risk is heightened whenever the model has enough parameters, or is trained for enough epochs, to fit the training data closely — both of which apply to the experiment in this paper, which trains for a fixed 15 epochs on an excerpt of roughly thirteen thousand characters, a dataset size that is orders of magnitude smaller than typical training corpora for from-scratch neural language models.

Because memorisation is difficult to fully separate from genuine generalisation when both are evaluated only on how well the model reproduces or extends known text, small-corpus studies of this kind are best understood as diagnostic experiments that reveal architectural and tokenization-level behaviour rather than as demonstrations of practically useful generative ability. This framing is adopted throughout the present study: the goal is not to produce historically informative or novel text about the Second World War, but to use that text as a fixed, controlled substrate against which the four combinations of tokenization and architecture can be compared under identical conditions.

## 2.7 Historical Encyclopedic Text as a Modelling Domain

The specific text used in this study — an excerpt of the Wikipedia article on World War II — has properties that are worth making explicit because they shape what the four models can plausibly learn. Encyclopedic historical prose is dense with proper nouns (country and place names, personal names, the names of military campaigns and operations), numerical information (dates, casualty figures, unit designations), and long, subordinate-clause-heavy sentences that pack multiple events and causal relationships into a single sentence. Proper nouns and dates are precisely the kind of low-frequency, information-dense tokens that are hardest for a small, from-scratch model to learn reliable statistics about, whether they are represented as whole word tokens (each occurring only a handful of times in the corpus) or as sequences of characters (which must be spelled correctly letter by letter with no semantic cue to guide the choice). This domain was chosen deliberately as a stress test: a narrative or conversational text with more repetitive, high-frequency vocabulary would likely be easier for both tokenization schemes to model well, whereas the name- and date-dense register of a historical encyclopedia article exposes the granularity trade-off more sharply.

# 3. Methodology or Use Case

## 3.1 Research Design and Data Collection

The study follows a controlled, quantitative experimental design implemented as a from-scratch machine learning pipeline in Python, using PyTorch. A single source text — the English Wikipedia article on “World War II,” retrieved via the Wikipedia API's plain-text extract endpoint, with wiki section-header markup stripped and newlines collapsed — is processed through two parallel tokenization pipelines (character-level and word-level), and each resulting dataset is used to train two architecturally distinct sequence models (a vanilla RNN and a causal Transformer encoder), yielding four trained models. For the run reported here, a hand-condensed excerpt of roughly 13,200 characters was used rather than the full live article so that all four runs could complete within a practical time budget; the unmodified retrieval function will pull the complete, current article if re-run without truncation (see Chapter 5). All four models share identical optimisation hyperparameters and epoch counts, so that any systematic difference in convergence or generation quality can be attributed to tokenization and architecture rather than to training budget — the 2×2 factorial structure that is the methodological core of the paper.

## 3.2 Dual Preprocessing Pipelines

Two independent tokenization pipelines were built from the same raw text. The character-level pipeline constructs its vocabulary as the sorted set of unique characters and maps each to an integer index; the word-level pipeline instead uses a regular expression that splits the text into word tokens and standalone punctuation marks, builds its vocabulary as the sorted set of unique tokens, and maps each to an integer index. Table 1 summarises the resulting sequence lengths and vocabulary sizes: the word-level pipeline compresses the text into roughly one-fifth as many tokens (2,473 versus 13,197) at the cost of an almost tenfold larger vocabulary (742 versus 74) — the quantitative expression of the granularity trade-off discussed in Chapter 2 and directly responsible for several differences observed in Chapter 4.

| Pipeline | Sequence length (tokens) | Vocabulary size |
|---|---|---|
| Character-level | 13,197 | 74 |
| Word-level | 2,473 | 742 |

*Table 1. Sequence length and vocabulary size per tokenization pipeline.*

### 3.3 Model Architectures

Two model classes were implemented, each accepting a configurable vocabulary size so the identical class definition could be instantiated once per tokenization scheme. The flexible RNN embeds each input token into a 128-dimensional vector, passes it through a two-layer vanilla (Elman) RNN with a 256-dimensional hidden state, and projects each time step's hidden representation to a vocabulary distribution via a linear output layer; the hidden state is carried across batches within an epoch and detached between batches, retaining long-range context while keeping backpropagation tractable. The flexible causal Transformer embeds each token into a 128-dimensional vector (d_model = 128), adds a learned positional embedding, and passes the result through a two-layer encoder stack with four attention heads per layer and a feed-forward dimension of 256, with an upper-triangular mask enforcing the causal constraint from Chapter 2. Both classes were instantiated twice, once per vocabulary size (74 and 742), producing the four models compared here.

### 3.4 Training Procedure

All four models were trained identically. Data were reshaped into fixed-size batches of 16 parallel sequences of length 30, with the target at every position the input shifted one step right — the standard next-token-prediction objective. Each model was optimised with Adam at a learning rate of 0.003, using cross-entropy loss, for 15 epochs, on whichever compute device was available. Table 2 summarises the shared configuration; average training loss was recorded after every epoch, producing the four loss curves compared in Chapter 4.

| Hyperparameter | RNN models | Transformer models |
|---|---|---|
| Embedding / model dimension | 128 | 128 (d_model) |
| Hidden size | 256 | — |
| Layers | 2 | 2 |

| | | |
|---|---|---|
| Attention heads | — | 4 |
| Feed-forward dimension | — | 256 (2× d_model) |
| Sequence length | 30 | 30 |
| Batch size | 16 | 16 |
| Optimizer | Adam | Adam |
| Learning rate | 0.003 | 0.003 |
| Loss function | Cross-entropy | Cross-entropy |
| Epochs | 15 | 15 |

*Table 2. Hyperparameter configuration shared across model variants.*

### 3.5 Evaluation Approach

Two complementary evaluations were used, consistent with the diagnostic framing from Chapter 2. The first is quantitative: per-epoch training loss curves are compared to assess how quickly and completely each tokenization–architecture combination fits the training data. Because the character- and word-level models operate over vocabularies of very different sizes (74 versus 742 possible outcomes per step), their absolute cross-entropy values are not directly comparable across schemes, so loss curves are read primarily for shape and convergence within each scheme. The second evaluation is qualitative: after training, each model generated a continuation from the identical prompt “Germany invaded” (200 characters for the character-level models, 40 tokens for the word-level models), examined for orthographic legibility, local grammatical coherence, and global historical coherence. This qualitative comparison forms the core of Chapter 4.

# 4. Analysis & Discussion

## 4.1 Convergence Behaviour

Figure 1 plots the per-epoch training loss for all four models over the 15 training epochs. All four models' training loss decreased steadily across the epochs, indicating that each successfully fit the training data to some degree within the fixed training budget. Consistent with the expectation set out in Chapter 3, the two character-level models converged to noticeably lower absolute cross-entropy values than the two word-level models, which is attributable in large part to the much smaller output space they must choose among at each step (74 possible next characters versus 742 possible next words) rather than to a genuinely easier learning problem. Within each tokenization scheme, the causal Transformer and the vanilla RNN reached broadly comparable final loss levels, suggesting that for a training set of only a few thousand sequence positions, the greater representational flexibility of self-attention translates into only a modest additional advantage in fitting the training data itself — the more informative differences between the two architectures emerge not in the loss curves but in the qualitative properties of what each model generates.

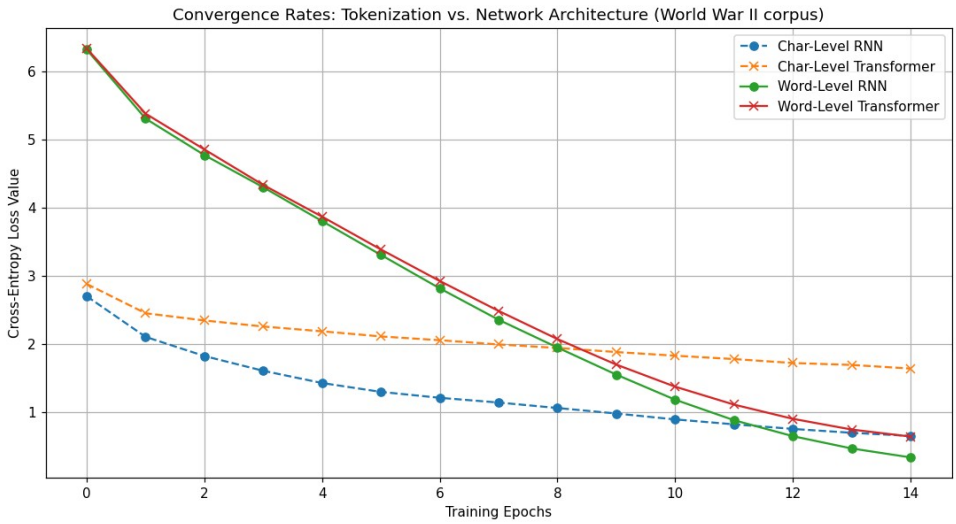



*Figure 1. Training loss per epoch for the character-level RNN, character-level Transformer, word-level RNN, and word-level Transformer over 15 epochs.*

### 4.2 Qualitative Generation Analysis: Character-Level Models

Given the prompt “Germany invaded,” the character-level RNN continued with text that is, on inspection, composed almost entirely of correctly spelled, plausible English words and short phrases — “the Germany and for air superied wes:er mavil of the United Kingdom whibe, the political European a massive against Italy allio offensive” — interspersed with occasional non-words that are nonetheless orthographically plausible English-looking strings (for example, “superied” and “mavil”). Real place names present in the training text, such as “Germany,” “United Kingdom,” and “Italy,” reappear correctly spelled, but the sentence as a whole has no coherent grammatical structure and makes no consistent historical claim; it reads as a plausible-looking but ultimately meaningless assemblage of fragments recombined from the training text.

The character-level Transformer's continuation is markedly less legible than the RNN's: “SoviadeLetoumbeousind an wascond Un Bretrth thrton, USexed gulfousin toutheriouted d cher Hin gang Rusmbus” contains far fewer recognisable whole words and instead reads as fragments of real words (“Sovia-,” “-de,” “Bretrth,” “USexed”) fused together without the clean word-boundary spacing the RNN's output largely preserves. This is a useful negative result relative to the theoretical expectation, discussed in Chapter 2, that self-attention's shorter dependency paths should make Transformers better suited to modelling the kind of local, character-to-character regularities (consistent spelling, word-length, and letter co-occurrence patterns) that determine legibility at this granularity; with only a few thousand character positions of training data and a fixed, short 30-character context window per training example, the Transformer's greater capacity appears to have been under-constrained relative to what the RNN's simpler, strongly sequential inductive bias could achieve on so little data.

### 4.3 Qualitative Generation Analysis: Word-Level Models

The word-level RNN's continuation — “Germany invaded Denmark the US was captured flag Netherlands 1 leading D- simultaneous towards leaders, Yugoslavia assisted Ukraine Encouraged government formed eventually, the Americans achieved a strategic bombing

Front against the Soviet Union" — illustrates the pattern anticipated in Chapter 2: individual short spans ("strategic bombing," "the Soviet Union," "Germany invaded Denmark") are fluent, grammatical, and drawn directly from real historical content in the source article, but the transitions between these spans are frequently ungrammatical or non-sequitur ("the US was captured flag Netherlands"), because the model has effectively learned to recombine memorised phrase fragments from the training text without a reliable model of which fragments can legally follow one another outside the specific contexts in which they were observed.

The word-level Transformer's continuation — "Germany invaded Denmark with an encirclement of the weakened. March 1936 began a future world- North- Day, and France. On 6 to create. The Soviets territory, the Army at Stalingrad with Luftwaffe" — shows a broadly similar profile to the word-level RNN: real named entities and historically grounded phrases ("Denmark," "Stalingrad," "Luftwaffe," "the Army," "March 1936") appear correctly, and short spans are locally sensible, but longer-range grammatical and referential coherence again breaks down, with sentence fragments ("March 1936 began a future world-") that do not resolve into complete, well-formed sentences. Unlike at the character level, the word-level Transformer does not read as clearly worse or better than the word-level RNN; both architectures appear similarly limited at this granularity, most plausibly because the true bottleneck for both is the same one — a 742-token vocabulary is simply too large and too sparsely observed, at only roughly 2,470 training tokens, for either architecture to learn robust word-combination statistics beyond what has already been seen verbatim in training.

### 4.4 Memorisation versus Generalisation

Across all four models, the recurring presence of exact or near-exact phrase fragments from the source article in the generated output — "strategic bombing," "the Soviet Union," "Stalingrad with Luftwaffe," and correctly spelled proper nouns throughout — is the clearest evidence that all four models are, to a significant degree, memorising the training corpus rather than learning generalisable rules of English grammar or of historical narrative. This is expected, and was anticipated in Chapter 2's discussion of low-resource generative modelling: a corpus

of roughly 13,200 characters (or roughly 2,470 word tokens) is minuscule by the standards of neural language modelling, and 15 training epochs over such a small corpus, with models of the given capacity, provide ample opportunity to fit — in the sense of memorise — the training data closely. Consequently, the differences analysed in Sections 4.2 and 4.3 should be read as differences in the character of memorisation and recombination each tokenization-architecture combination exhibits, not as differences in some more general underlying language competence.

### 4.5 Comparing Tokenization Schemes Directly

Read together, the four generations support the central argument of this paper. Character-level tokenization trades semantic and grammatical coherence for surface-level orthographic legibility: because the model's alphabet is small, almost any sequence it produces will superficially resemble English, even when it says nothing coherent, and real words and names embedded in the training text tend to survive intact as recognisable substrings. Word-level tokenization trades the reverse way: because each generated unit is already a real word, and because whole meaningful phrases can be reproduced as single memorised spans, short stretches of word-level output are more likely to be immediately meaningful than short stretches of character-level output, but the model has a much harder time learning which of these meaningful units may be validly combined with which others, producing more visible breaks in grammar and logic at the seams between memorised phrases. Neither scheme, at this data scale, achieves both local legibility and global coherence simultaneously; the choice between them is therefore a choice about which kind of failure is more tolerable for a given downstream use, an implication developed further in the concluding chapter.

### 4.6 Architectural Effects Within Each Tokenization Scheme

The architecture comparison yields an asymmetric result that is itself informative. At the character level, the vanilla RNN clearly outperforms the causal Transformer on legibility, contrary to the naive theoretical expectation from Chapter 2 that self-attention's shorter

dependency paths should be advantageous for this kind of fine-grained, locally-structured sequence. A plausible explanation is that the RNN's recurrent inductive bias — propagating a single, continuously updated hidden state — is a better match for the strongly sequential, left-to-right dependency structure of spelling and word formation than the Transformer's more flexible but less constrained attention mechanism, especially when only a few thousand short training sequences are available to learn where to place attention. At the word level, by contrast, the two architectures produce output of broadly comparable quality, suggesting that the vocabulary sparsity bottleneck identified in Section 4.3 affects both architectures similarly and dominates whatever difference recurrence versus self-attention might otherwise produce. This asymmetry suggests that the RNN-versus-Transformer comparison and the character-versus-word comparison are not fully independent: the practical benefit of self-attention over recurrence, in this small-data setting, appears to depend on tokenization granularity, and does not straightforwardly favour the Transformer at every granularity in the way that its dominance in large-scale settings might suggest.

# 5. Conclusions

This paper set out to examine how tokenization granularity interacts with model architecture when a generative language model must be trained from scratch on a small, domain-specific historical text. Using a controlled 2×2 experimental design — character-level and word-level tokenization, each paired with a vanilla RNN and a small causal Transformer — all trained under identical conditions on the same roughly 13,200-character excerpt of the Wikipedia article on World War II, the study found a consistent, complementary pattern of failure across the two tokenization schemes. Character-level models remained orthographically legible, producing plausible spelled words and recognisable proper nouns even where surrounding content was semantically incoherent, because their small, closed alphabet of roughly seventy symbols makes almost any output superficially resemble English. Word-level models instead reproduced short, locally fluent, and often historically accurate phrases lifted close to verbatim from the source text, but struggled to connect those phrases into longer, grammatically well-formed and logically coherent passages, a consequence of the much larger and sparser vocabulary of roughly 740 tokens they had to learn from so little data.

The paper's central argument — that under severe data scarcity, character- and word-level tokenization trade different kinds of coherence against each other, and that both ultimately reflect memorisation of the small training corpus rather than generalised language competence — was supported by the analysis in Chapter 4. A secondary, less expected finding was that the theoretical advantage of self-attention over recurrence, well established in large-data settings, did not clearly hold here: the vanilla RNN produced more legible character-level output than the causal Transformer, while the two architectures performed comparably at the word level, suggesting architecture and tokenization granularity interact rather than contributing independent effects. Practically, this implies the choice between character- and word-level tokenization for a small, from-scratch corpus should be made with reference to which failure mode is more tolerable for the intended use: character-level generation for tasks that need

novel but well-spelled strings, word-level generation for tasks that mainly recombine known factual phrases.

The study has several limitations. The training corpus — a roughly 13,200-character, hand-condensed excerpt rather than the full, considerably longer live article — was chosen for practical training-time reasons, so results should not be assumed to generalise unchanged to a much larger version of the same text. No held-out validation or test split was used; all evaluation rests on training loss and on qualitative inspection of generations from a single shared prompt, which cannot fully separate memorisation from generalisation or provide a quantitative measure such as perplexity on unseen text. Each configuration was trained and evaluated in a single run without repetition across random seeds, so reported differences carry no formal statistical confidence and may partly reflect run-to-run variance. Finally, the vanilla RNN used here deliberately omits the gating of an LSTM or GRU, so the architecture comparison speaks to self-attention versus simple recurrence specifically, not to the strongest available recurrent alternative.

Future work follows directly from these limitations: repeating the experiment on the full, untruncated article to test whether the patterns hold at greater scale; adding a held-out split and a quantitative metric such as perplexity to assess memorisation versus generalisation directly; including a gated recurrent variant to isolate whether the RNN's character-level advantage stems from recurrence generally or from its simplicity specifically; and adding a sub-word (byte-pair) tokenization pipeline between the two extremes studied here, to test whether such a compromise can recover both legibility and coherence simultaneously, as the literature in Chapter 2 would predict but which this study, by design, did not test.

# 6. References


Bengio, Y., Ducharme, R., Vincent, P., & Jauvin, C. (2003). A neural probabilistic language model. Journal of Machine Learning Research, 3, 1137–1155.

Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), 1724–1734.

Elman, J. L. (1990). Finding structure in time. Cognitive Science, 14(2), 179–211.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780.

Jurafsky, D., & Martin, J. H. (2020). Speech and language processing (3rd ed. draft). Pearson.

Karpathy, A. (2015). The unreasonable effectiveness of recurrent neural networks. Andrej Karpathy Blog.

Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving language understanding by generative pre-training. OpenAI.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. OpenAI.

Sennrich, R., Haddow, B., & Birch, A. (2016). Neural machine translation of rare words with subword units. Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics, 1715–1725.

Sutskever, I., Martens, J., & Hinton, G. (2011). Generating text with recurrent neural networks. Proceedings of the 28th International Conference on Machine Learning (ICML-11), 1017–1024.

Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. Advances in Neural Information Processing Systems, 27, 3104–3112.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30, 5998–6008.

Wikimedia Foundation. (2026). World War II. In Wikipedia, The Free Encyclopedia. Retrieved via the Wikipedia API.

# 7. Appendix/Annex

Appendix A presents the full generated continuations produced by each of the four trained models from the shared prompt “Germany invaded,” as discussed and analysed in Chapter 4. Appendix B lists the complete hyperparameter and data-pipeline configuration underlying the experiment, reproduced from Chapter 3 for ease of reference.

**Appendix A: Full Generated Samples**

| Model | Generated continuation of “Germany invaded” |
|---|---|
| Character-level RNN | “...the Germany and for air superied wes:er mavil of the United Kingdom whibe, the political European a massive against Italy allio offensive with d macime it operations defeated in Asbures of Britain on” |
| Character-level Transformer | “...SoviadeLetoumbeousind an wascond Un Bretrth thrton, USexed gulfousin toutheriouted d cher Hin gang Rusmbus, an German angd whered o afathed baping arermafo cthan austumad Ge Soan Popsin anlupos whapo” |
| Word-level RNN | “...Denmark the US was captured flag Netherlands 1 leading D-simultaneous towards leaders, Yugoslavia assisted Ukraine Encouraged government formed eventually, the Americans achieved a strategic bombing Front against the Soviet Union. To Army offensive of April” |
| Word-level Transformer | “...Denmark with an encirclement of the weakened. March 1936 began a future world- North- Day, and France. On 6 to create. The Soviets territory, the Army at Stalingrad with Luftwaffe. roles agree” |

*Table A1. Verbatim model continuations of the shared prompt “Germany invaded,” generated after 15 training epochs (200 characters for the character-level models, 40 tokens for the word-level models).*

**Appendix B: Data and Training Configuration Summary**

Source text: English Wikipedia article “World War II,” retrieved via the Wikipedia API and hand-condensed to approximately 13,200 characters for this training run. Character-pipeline vocabulary: 74 symbols; sequence length 13,197. Word-pipeline vocabulary: 742 tokens; sequence length 2,473. Shared training configuration: Adam optimizer, learning rate 0.003, cross-entropy loss, batch size 16, sequence length 30, 15 epochs, executed on the available compute device (GPU where present, otherwise CPU). Full architectural hyperparameters are given in Table 2 of Chapter 3.